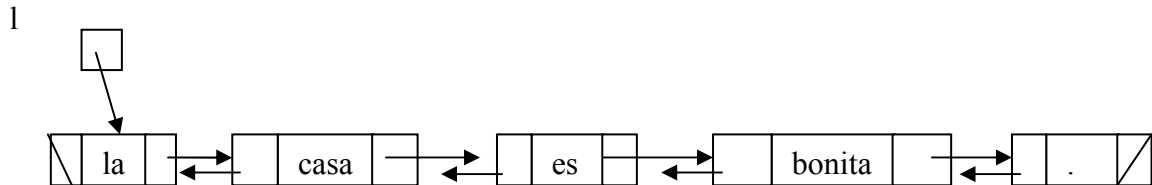


ESTRUCTURAS DE DATOS Y ALGORITMOS

ENERO 2012

1. (1,5 puntos) Tenemos una lista doblemente ligada que contiene una lista de palabras:



Se quiere implementar la siguiente función:

```
public DoubleNode<T> {
    T data;
    DoubleNode<T> next;
    DoubleNode<T> prev;
}

public DoubleLinkedList<T> {
    DoubleNode<T> first;
}

public ListaDePalabras extends DoubleLinkedList<String> { // herencia

    public SimpleLinkedList<String> obtenerContexto(String pal, Integer n)
    // pre: la palabra "pal" se encuentra en la lista
    //      n >= 0
    // post: devuelve una lista que contiene la palabra buscada junto con
    //       las "n" palabras anteriores y posteriores a "pal"
    //       En caso de que la palabra "pal" aparezca varias veces, se tendrá
    //       en cuenta únicamente la primera aparición
    //       En caso de que no haya n palabras delante o detrás de "pal"
    //       se devolverán solamente las que haya
}
```

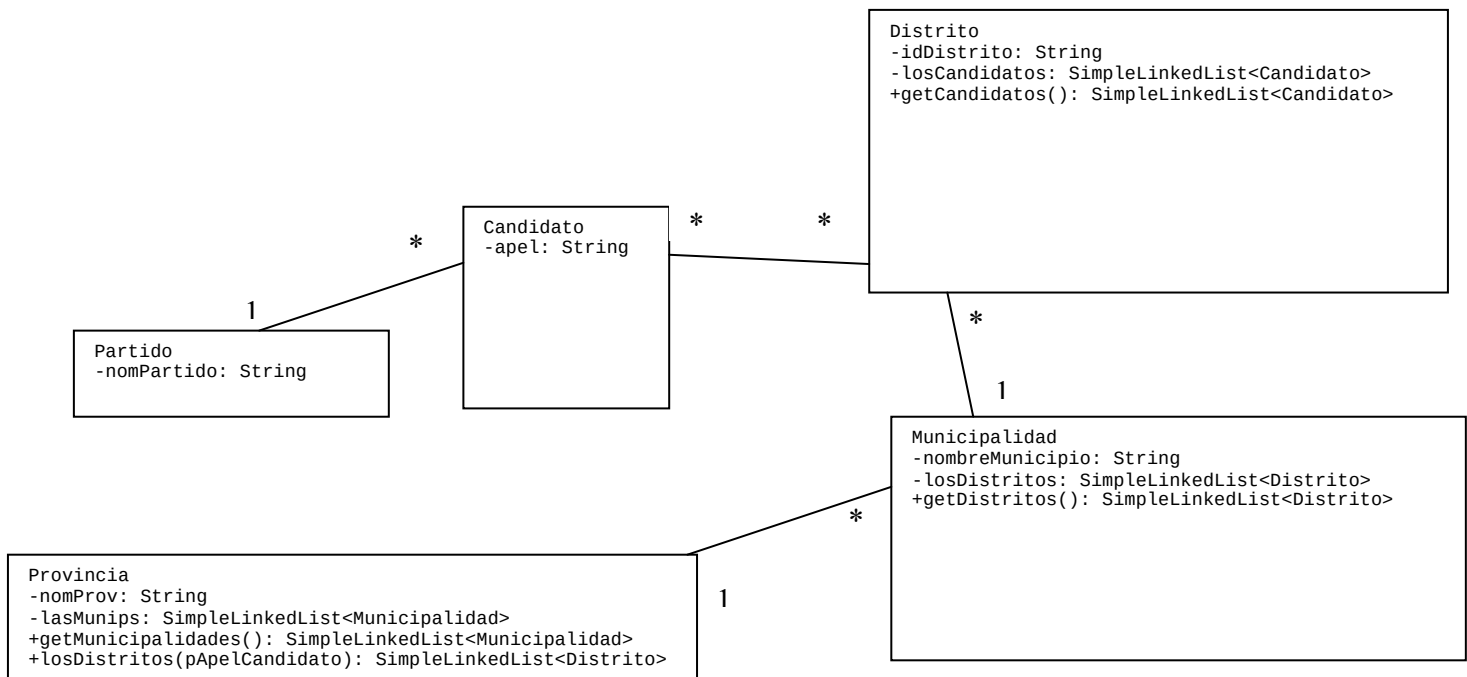
Por ejemplo, la llamada a `obtenerContexto("es", 1)` devolverá la lista `<"casa", "es", "bonita">`.

Se pide:

- Implementar el algoritmo
 - Establecer de manera razonada el coste del algoritmo.
-

2. Elecciones (1,5 puntos)

Dado el siguiente diagrama de clases:



Y dada la siguiente clase:

```
public class Candidatura {
    String partido;
    String apelCandidato;
    String idDistrito;
    String municipio;
    String prov;
}
```

Se pide implementar el siguiente método de la clase Eleccion, calculando **razonadamente** su coste:

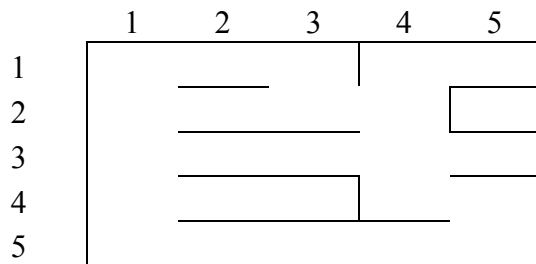
```
public class Eleccion {

    SimpleLinkedList<Partido> partidos; // todos los partidos
    SimpleLinkedList<Candidato> candidatos; // todos los candidatos
    SimpleLinkedList<Distrito> distritos; // todos los distritos
    SimpleLinkedList<Municipio> municipios; // todos los municipios
    SimpleLinkedList<Provincia> provincias; // todas las provincias

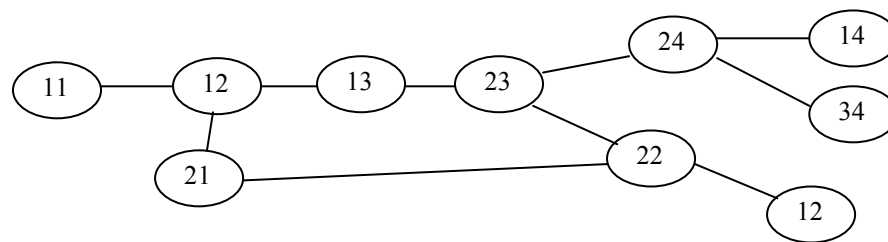
    public añadirCandidaturas(SimpleLinkedList<Candidatura> lc)
    // Pre: las listas de partidos, candidatos, distritos,
    // municipios y provincias se encuentran vacías
    // Post: se han añadido las candidaturas dadas, actualizando
    // todos los datos
}
```

3. Laberinto (1,5 puntos)

La siguiente figura muestra un ejemplo de un laberinto.



Se puede representar el laberinto por medio de un grafo, que presentamos parcialmente en la siguiente figura:



Cada nodo representa una casilla y queda identificado por un string que indica la fila y la columna (por ejemplo, el nodo “23” indica que corresponde a la fila 2, columna 3). El grafo contiene un arco entre dos nodos si corresponden a dos casillas consecutivas y es posible pasar de una a otra.

Queremos desarrollar un algoritmo que, dados dos puntos del laberinto, nos devuelva un camino de longitud mínima que conecta esos dos puntos. El algoritmo devolverá:

- La lista vacía si no hay ningún camino que los une
- En caso de que haya varios caminos de longitud mínima, bastará con devolver cualquiera de ellos

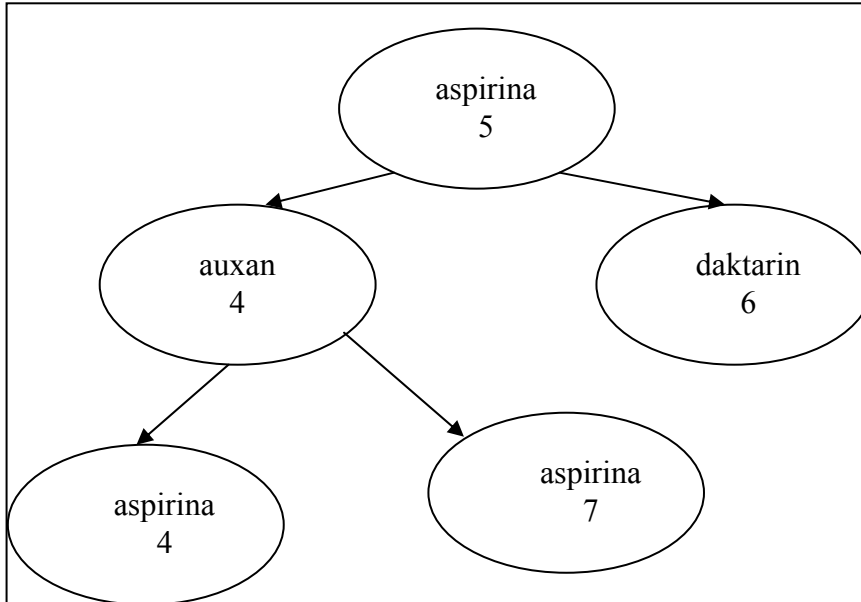
Por ejemplo, la llamada `buscarCamino(“11”, “35”)` devolverá la lista `<11, 12, 13, 23, 24, 34, 35>` (obsérvese que hay otros caminos de longitud igual o superior a este).

```
public class GraphAL<T> implements GraphADT<T>
{
    protected final int DEFAULT_CAPACITY = 100;
    protected int numVertices; // number of vertices in the graph
    protected LinkedList<Integer>[] adjList; // adjacency list
    protected T[] vertices; // values of vertices
}

public class GrafoLaberinto extends GraphAL<String> {
    public SimpleLinkedList<String> buscarCamino(String inicio, String fin)
}
```

4. Conversión de árbol a tabla hash (1,5 puntos)

Dado un árbol binario que contiene datos de ventas de productos (en cada nodo aparece un par del tipo (producto, unidades vendidas)), queremos un subprograma que vuelque los elementos del árbol a una tabla hash, obteniendo un solo elemento para cada producto, cuyo valor asociado será el total de las ventas de ese producto.



Dado el árbol ejemplo, la tabla hash resultante contendrá el siguiente resultado:

0		
1	daktarin	6
2		
3	aspirina	16
4	auxan	4
5		

```
public class BinaryTreeNode<T> {
    protected T content;
    protected BinaryTreeNode<T> left;
    protected BinaryTreeNode<T> right;
}
public class BinaryTree<T> {
    protected int count;
    protected BinaryTreeNode<T> root;
}

public class Medicina {
    int ventas;
    String nombre;
}

public class ArbolMedicinas extends BinaryTree<Medicina> {

    public HashMap<String, Integer> convertirArbolEnTH()
    // Post: el resultado es una table hash con los elementos del árbol, donde
    //      cada elemento aparece una sola vez (el dato asociado a cada producto
    //      es el total de ventas de ese producto)
}
```

Lista

```
public interface ISimpleLinkedList<T> {
    public void goFirst();
    public void goNext();
    public void goPrevious();
    public void goLast();
    public boolean hasNext();
    public boolean find(T elem);
    public void insert(T elem);
    public void insertFirst(T elem);
    public void insertLast(T elem);
    public T remove(int index);
        // Pre: al menos hay "index" elementos
    public void remove();
    public void remove(T elem);
    public T get();
    public T get(int index);
        // Pre: al menos hay "index" elementos
    public boolean isEmpty();
}
```

Pila

```
public interface StackADT<T> {
    public void push(T elem)
    public T pop()
    public T peek()
    public boolean isEmpty()
    public int size()
}
```

Ilara / Cola

```
public interface QueueADT<T> {
    public void insert(T elem)
    public T remove()
    public T first()
    public boolean isEmpty()
    public int size()
}
```

Hash taula / Tabla hash

```
public interface Map<K, V> {
    public V put(K key, V value)
    public V get(K key)
    public T remove(K key)
    public boolean containsKey(K key)
    public int size()
}
```